Assignment-16.2

Name: Adepu Tejaswini

Ht.no: 2303A51423

Batch. No: 21

Task Descrption-1: (Design Database Schema for Student Management System)

Instructions:

• Using AI tools, generate CREATE TABLE statements for a Student Management Database.

• Tables: Students, Courses, Enrollments.

• Include primary keys, foreign keys, and suitable data types. Ensure the schema follows normalization rules (1NF, 2NF, 3NF).

Sample Code:

```
CREATE TABLE Students (
student_id INT PRIMARY KEY,
name VARCHAR(100),
email VARCHAR(100)
);
CREATE TABLE Courses (
course_id INT PRIMARY KEY,
course_name VARCHAR(100),
credits INT
);
CREATE TABLE Enrollments (
```

enrollment_id INT PRIMARY KEY,

student_id INT,

course_id INT,

FOREIGN KEY (student_id) REFERENCES Students(student_id),

FOREIGN KEY (course_id) REFERENCES Courses(course_id)

);

Prompt:

Generate SQL CREATE TABLE statements for a Student Management System with tables: Students, Courses, and Enrollments.

Add primary keys, foreign keys, and correct data types.

Make sure the design follows normalization (1NF, 2NF, 3NF) and is simple and clear.

Code:

```sql
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    credits INTEGER NOT NULL
);
CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    enrollment_date DATE NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Output:

```sql
CREATE TABLE Students (
student_id INTEGER PRIMARY KEY AUTOINCREMENT,
first_name TEXT NOT NULL,
last_name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL
);
```

```sql
CREATE TABLE Courses (
course_id INTEGER PRIMARY KEY AUTOINCREMENT,
course_name TEXT NOT NULL,
credits INTEGER NOT NULL
);
```

```sql
CREATE TABLE Enrollments (
enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id INTEGER NOT NULL,
course_id INTEGER NOT NULL,
enrollment_date DATE NOT NULL,
FOREIGN KEY (student_id) REFERENCES Students(student_id),
FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Observation:

The code creates tables with proper primary keys and foreign key connections between them.
It follows normalization rules, so there is no repeated or unnecessary data.
The database structure is simple, well organized, and easy to understand and maintain.

Task Description- 2: (Insert Sample Records)

Instructions:

• Use AI tools to generate INSERT statements to populate the

tables.

• Insert at least 3 students, 4 courses, and 5 enrollment records.

Verify data using SELECT queries.

Sample Code:

```
INSERT INTO Students VALUES
(1, 'Arjun', 'arjun@gmail.com'),
(2, 'Meera', 'meera@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');
INSERT INTO Courses VALUES
(101, 'Database Systems', 4),
(102, 'Data Structures', 3),
(103, 'Operating Systems', 4),
(104, 'AI Fundamentals', 3);
INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 1, 102),
(3, 2, 103),
(4, 3, 101),
(5, 3, 104);
```

Prompt:

Generate SQL INSERT statements to add sample data into Students, Courses, and Enrollments tables.

Insert at least 3 students, 4 courses, and 5 enrollment records.

Also write SELECT queries to display all records from each table.

Code:

```sql
INSERT INTO Students (first_name, last_name, email) VALUES ('John', 'Doe', 'john.doe@example.com'),
('Jane', 'Smith', 'jane.smith@example.com'),
('Bob', 'Johnson', 'bob.johnson@example.com');
INSERT INTO Courses (course_name, credits) VALUES
('Mathematics', 3),
('Physics', 4),
('Chemistry', 3),
('Computer Science', 3);
INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES (1, 1, '2023-01-01'),
(1, 2, '2023-01-01'),
(2, 1, '2023-01-01'),
(2, 3, '2023-01-01'),
(3, 2, '2023-01-01');
SELECT * FROM Students;
SELECT * FROM Courses;
SELECT * FROM Enrollments;
```

Output:

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO Students (first_name, last_name, email) VALUES ('John',
'Doe', 'john.doe@example.com'),
('Jane', 'Smith', 'jane.smith@example.com'),
('Bob', 'Johnson', 'bob.johnson@example.com');
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO Courses (course_name, credits) VALUES
('Mathematics', 3),
('Physics', 4),
('Chemistry', 3),
('Computer Science', 3);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
INSERT INTO Enrollments (student_id, course_id, enrollment_date)
VALUES (1, 1, '2023-01-01'),
(1, 2, '2023-01-01'),
(2, 1, '2023-01-01'),
(2, 3, '2023-01-01'),
(3, 2, '2023-01-01');
```

SQL ▾ 1 / 1 1 - 3 of 3

```
SELECT * FROM Students;
```

student_id	first_name	last_name	email
1	John	Doe	john.doe@example.com
2	Jane	Smith	jane.smith@example.com
3	Bob	Johnson	bob.johnson@example.com

SQL ▾ 1 / 1 1 - 4 of 4

```
SELECT * FROM Courses;
```

course_id	course_name	credits
1	Mathematics	3
2	Physics	4
3	Chemistry	3
4	Computer Science	3

SQL ▾ 1 / 1 1 - 5 of 5

```
SELECT * FROM Enrollments;
```

enrollment_id	student_id	course_id	enrollment_date
1	1	1	2023-01-01
2	1	2	2023-01-01
3	2	1	2023-01-01
4	2	3	2023-01-01
5	3	2	2023-01-01

Go Live

Observation:

The code inserts sample data into all tables successfully.
It uses SELECT queries to check and display the inserted records.
The structure is simple and helps to easily verify and understand the data.

Task Description- 3: (Perform CRUD Operations)

Instructions:

Use AI to generate SQL queries for basic CRUD operations:

- Retrieve all courses
- Update a student's email
- Delete an enrollment record
- Validate the changes using SELECT statements.

Sample Code:

SELECT * FROM Courses;

UPDATE Students

SET email = 'arjun_new@gmail.com'

WHERE student_id = 1;

DELETE FROM Enrollments

WHERE enrollment_id = 5;

Prompt:

Generate SQL queries to perform **CRUD operations** on a Student Management Database.
Include queries to **retrieve all courses, update a student email, and delete an enrollment record**.
Also add **SELECT queries to verify the changes**, keeping the code simple and clear.

Code:

```
-- Retrieve all courses
SELECT * FROM Courses;
-- Update a student's email
UPDATE Students
SET email = 'john.new@example.com'
WHERE student_id = 1;
-- Validate the update
SELECT * FROM Students WHERE student_id = 1;
-- Delete an enrollment record
DELETE FROM Enrollments
WHERE enrollment_id = 5;
-- Validate the deletion
SELECT * FROM Enrollments WHERE enrollment_id = 1;
```

Output:

SQL ▾ 1 / 1 1 - 4 of 4

```
SELECT * FROM Courses;
```

course_id	course_name	credits
1	Mathematics	3
2	Physics	4
3	Chemistry	3
4	Computer Science	3

SQL ▾ 1 / 1 1 - 0 of 0

```
UPDATE Students
SET email = 'john.new@example.com'
WHERE student_id = 1;
```

SQL ▾ 1 / 1 1 - 1 of 1

```
SELECT * FROM Students WHERE student_id = 1;
```

student_id	first_name	last_name	email
1	John	Doe	john.new@example.com

SQL ▾ 1 / 1 1 - 0 of 0

```
DELETE FROM Enrollments
WHERE enrollment_id = 5;
```

SQL ▾ 1 / 1 1 - 1 of 1

```
SELECT * FROM Enrollments WHERE enrollment_id = 1;
```

enrollment_id	student_id	course_id	enrollment_date
1	1	1	2023-01-01

Observation:

The code performs basic CRUD operations like select, update, and delete.
It updates the student email and removes the selected enrollment record correctly.
The SELECT queries help to verify the changes and ensure the data is correct.

Task Description- 4: (Execute Aggregate Queries)

Instructions:

Use AI tools to write aggregate queries:

• Count number of students enrolled per course

• Display courses with more than one student enrolled

Analyze the correctness of results.

Sample Code:

SELECT c.course_name, COUNT(e.student_id) AS student_count

FROM Courses c

JOIN Enrollments e ON c.course_id = e.course_id

GROUP BY c.course_name;

SELECT c.course_name

FROM Courses c

JOIN Enrollments e ON c.course_id = e.course_id

GROUP BY c.course_name

HAVING COUNT(e.student_id) > 1;

Prompt:

Generate SQL aggregate queries for a Student Management Database.

Write queries to count number of students per course and show courses with more than one student enrolled.

Use JOIN, GROUP BY, and HAVING, and keep the queries simple and clear.

Code:

```
-- Count number of students per course
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
-- Show courses with more than one student enrolled
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
```

Output:

SQL ▾ 1 / 1 1 - 4 of 4

```sql
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
```

course_name	student_count
Chemistry	1
Computer Science	0
Mathematics	2
Physics	1

SQL ▾ 1 / 1 1 - 1 of 1

```sql
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
```

course_name	student_count
Mathematics	2

Observation:

The code uses aggregate functions to count students in each course.
It correctly shows courses that have more than one student enrolled.
The queries are simple and help to analyze the data easily.

Task Description- 5: (Query Optimization)

Instructions:

• Use AI tools to analyze query performance.

• Rewrite inefficient sub queries using JOINs and test both queries

for correctness.

Sample Code:

-- Original Query

SELECT * FROM Students

WHERE student_id IN (

SELECT student_id FROM Enrollments WHERE course_id = 101

);

-- Optimized Query

SELECT s.*

FROM Students s

JOIN Enrollments e ON s.student_id = e.student_id

WHERE e.course_id = 101;

Prompt:

Analyze a SQL query with a subquery and rewrite it using a JOIN for better performance.

Show both the original query and optimized query, and ensure both give the same result.

Keep the queries simple and easy to understand.

Code:

```
-- Original Query
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 1
);
-- Optimized Query
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 1;
```

Output:

SQL ▾ 1 / 1 1 - 2 of 2

```sql
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 1
);
```

student_id	first_name	last_name	email
1	John	Doe	john.new@example.com
2	Jane	Smith	jane.smith@example.com

SQL ▾ 1 / 1 1 - 2 of 2

```sql
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 1;
```

student_id	first_name	last_name	email
1	John	Doe	john.new@example.com
2	Jane	Smith	jane.smith@example.com

Observation:

The code replaces a subquery with a JOIN to improve performance.
Both queries return the same data, so the result is correct.
The optimized query is faster and more efficient, especially for large data.